

NetworkPeer-Integration-Deployment-Guide

NetworkPeer Feature, Code, Integration, and Deployment Guide

This guide describes the implemented backend through Phase 9, the current web platform, the exact code ownership of each feature, and the ordered integration and deployment path.

1. Repositories and Boundaries

Repository	Path	Responsibility
API	NetworkPeer-main	Fastify API, PostgreSQL/PostGIS schema, Redis auth/rate limits, S3 evidence, Socket.IO, sync, notifications, optional FCM delivery.
Web platform	NetworkPeer-platform-main	TanStack Start/React website for client, worker, and admin experiences.

The backend is authoritative. Frontend state is a cache and rendering layer only. Authorization, ownership, lifecycle transitions, evidence acceptance, notification creation, and sync ordering remain in PostgreSQL and backend services.

2. Implemented Feature Map

Phase	User-visible feature	Backend logic
1	Phone OTP sign-in, sessions, roles	Redis OTPs, HMAC JWTs, refresh-family rotation, DB user/profile invariants
2	Auth protection and worker/client roles	<code>requireAuth</code> , current-user lookup, <code>requireRole</code> , Redis rate limits
3	Client job management	Atomic create with subtasks, ownership-scoped list/detail/cancel
4	Worker discovery and acceptance	Privacy-safe public projections, geospatial search, atomic acceptance
5	Worker progress and evidence	Worker lifecycle endpoint, S3 POST reservation, object verification, version-pinned evidence, submit validation
6	Live sync and notifications	Durable outbox, commit-ordered per-user cursors, Socket.IO user rooms, notification inbox, worker sync bootstrap, device registration, optional FCM HTTP v1 delivery
7	Admin backoffice and recovery controls	Append-only admin audit log, audited job overrides, user suspension, worker verification, worker

Phase	User-visible feature	Backend logic
		SQLite sync, and currency-safe analytics
8	Escrow, wallet, and payouts	Webhook-settled escrow holds, immutable double-entry journals, payout dispatch, wallet summaries, and payment-operation idempotency
9	Background queues	PostgreSQL media/sync outboxes relayed through BullMQ, version-pinned media processing, and retry-safe push delivery

3. Backend Code Map

3.1 Application Core

File	What it does	Website relationship
<code>src/index.ts</code>	Creates Fastify, CORS, routes, Socket.IO hub, push dispatcher, graceful shutdown.	The web app calls the REST prefix and Socket.IO path mounted here.
<code>src/background-worker.ts</code>	Starts background queues and payment dispatch without binding an HTTP listener.	Use for separately scaled workers.
<code>src/config.ts</code>	Validates all environment variables, production TLS, CORS, S3, push, and sync bounds.	Deployment values for web origin and API behavior are enforced here.

File	What it does	Website relationship
<code>src/db.ts</code>	Owns PostgreSQL, Redis, and isolated admin/media/financial PostgreSQL pools.	Every API and realtime request reaches these shared resources; privileged production workflows use dedicated database principals.
<code>src/contracts.ts</code>	Backend TypeScript domain/API shapes: users, jobs, media, sync events, notifications, envelopes.	The frontend manually maintains a partial auth/sync/notification subset in its sibling <code>src/lib/api.ts</code> ; job and media contracts are not shared code.
<code>src/repository.ts</code>	Sole raw-SQL/PostGIS data layer.	All website-facing data is read or changed through service calls into this file.
<code>src/state-machine.ts</code>	Pure legal job transition table.	Explains the worker timeline rendered by the web UI.
<code>src/auth.ts</code>	OTP hashing, JWT signing/verification, refresh-family rotation, Redis rate helpers.	Powers the phone code workflow and browser access token.

3.2 Middleware and Utilities

File	What it does
<code>src/middleware/auth.ts</code>	Validates bearer JWTs, reloads active/verified users, supplies <code>request.auth</code> , enforces roles.
<code>src/middleware/rate-limit.ts</code>	Redis-backed IP rate limit; <code>/live</code> is intentionally exempt for liveness checks.
<code>src/utils/validation.ts</code>	Converts Zod body failures into typed, safe API errors.

3.3 Route Files

File	REST surface	Website page
<code>src/routes/system.ts</code>	<code>/</code> , <code>/live</code> , <code>/health</code>	Service identi deployment p
<code>src/routes/auth.ts</code>	OTP request/verify, refresh, logout, <code>/auth/me</code>	<code>/auth</code> , <code>/au</code> session refres
<code>src/routes/client-jobs.ts</code>	Client create/list/detail/cancel jobs	Planned repla <code>/client/job</code>
<code>src/routes/worker-jobs.ts</code>	Nearby jobs, protected detail, accept	Planned repla <code>/worker</code> , <code>/worker/job</code>
<code>src/routes/admin-workers.ts</code>	Worker verification updates	Planned repla admin worker screen.
<code>src/routes/admin.ts</code>	Admin audit log, job overrides, user management/suspension, and analytics	Planned repla admin jobs, w clients, and a screens.
<code>src/routes/work.ts</code>	Worker status, S3 upload URL, evidence confirmation, submit	Planned repla <code>/worker/ta</code>
<code>src/routes/sync.ts</code>	<code>GET /sync?cursor=</code>	Used by <code>RealtimeSyr</code> after connection/re
<code>src/routes/worker-sync.ts</code>	<code>GET /worker/sync?</code> <code>cursor=</code>	Durable work assigned-job endpoint for a client.
<code>src/routes/notifications.ts</code>	Notification list/read/read-all/device registration	Used by <code>/client/not</code> device registr for a Firebase

File	REST surface	Website page
<code>src/routes/financial.ts</code>	Escrow funding, client approval, and client/worker wallets	Planned replacement client review and wallet screen
<code>src/routes/payment-webhooks.ts</code>	Raw-byte signed payment webhook ingestion	Called only by configured payment provider.

3.4 Service Files

File	What it does
<code>src/services/auth-service.ts</code>	Creates/fetches phone users after OTP verification and issues token pairs.
<code>src/services/otp-service.ts</code>	Controls OTP TTL, attempt limits, and SMS delivery.
<code>src/services/sms-provider.ts</code>	Console/Twilio SMS provider boundary.
<code>src/services/job-service.ts</code>	Client job policy above SQL.
<code>src/services/worker-job-service.ts</code>	Verified-worker discovery, privacy and acceptance policy.
<code>src/services/admin-worker-service.ts</code>	Admin verification policy and busy-worker conflict mapping.
<code>src/services/admin-service.ts</code>	Audited admin override, suspension, audit-log, user-management, and analytics policy.
<code>src/services/media-storage-service.ts</code>	S3 POST policy, <code>HeadObject</code> , tagging, and production bucket readiness validation.
<code>src/services/work-evidence-service.ts</code>	Worker progress, evidence reservation/confirmation, version pinning, submit policy.
<code>src/services/notification-service.ts</code>	Cursor sync, notification paging/read state, device token registration.

File	What it does
<code>src/services/realtime-hub.ts</code>	Authenticates Socket.IO connections, joins user rooms, listens to PostgreSQL <code>NOTIFY</code> , emits local realtime events.
<code>src/services/push-notification-service.ts</code>	Durable push delivery worker and dependency-free FCM HTTP v1 JWT/OAuth sender.
<code>src/services/ledger-service.ts</code>	Funding, approval, payout dispatch/retry, webhook settlement, and wallet policy.
<code>src/services/payment-gateway-service.ts</code>	Stub and Stripe Connect adapters plus raw Stripe-style signature verification.
<code>src/services/payment-dispatch-service.ts</code>	PostgreSQL-leased retry dispatcher for payment operations whose gateway call was interrupted.
<code>src/services/background-queue-service.ts</code>	BullMQ relay/workers for durable media-processing and push-delivery outboxes.

3.5 Financial Workflow

Files	What exists
<code>migrations/021_add_funding_job_status.sql</code> through <code>035_restore_admin_audit_transition_rejection.sql</code> , <code>src/routes/financial.ts</code> , <code>src/routes/payment-webhooks.ts</code> , <code>src/services/ledger-service.ts</code> , <code>src/services/payment-gateway-service.ts</code>	A new job starts <code>FUNDING / UNFUN</code> a successful sign webhook creates balanced escrow and publishes it a <code>POSTED / HELD</code> . Approval atomica releases escrow, records platform revenue, creates pending payout journal, and dispatches the external payout.

Files	What exists
	Wallet balances a projections of completed immut postings.

3.6 Migrations, Tests, and Tooling

Files
migrations/001_enable_postgis_and_enums.sql through 005_enforce_job_lifecycle.sql
migrations/006_harden_worker_admission_and_geospatial_search.sql through 010_repair_nearby_index_and_worker_admission.sql
migrations/011_add_phase_5_media_evidence.sql through 016_harden_s3_version_pinning_and_submission.sql
migrations/017_add_phase_6_sync_and_notifications.sql through 019_harden_phase_6_sync.sql
migrations/018_harden_phase_1_to_5_integrity.sql

Files

migrations/020_add_phase_7_admin_backoffice.sql

migrations/021_add_funding_job_status.sql through
035_restore_admin_audit_transition_rejection.sql

migrations/036_harden_payment_reversals_and_dispatch.sql

migrations/036_add_phase_9_media_processing_outbox.sql

migrations/037_fix_payment_dispatch_name_resolution.sql

Files
<code>scripts/migrate.ts</code>
<code>scripts/provision-app-role.sql</code>
<code>scripts/provision-admin.sql</code>
<code>scripts/verify-phase1.ts</code>
<code>scripts/verify-auth-security.ts</code>
<code>scripts/verify-phase4.ts</code>
<code>scripts/verify-phase5.ts</code>
<code>scripts/verify-phase6.ts</code>
<code>scripts/verify-phase7.ts</code>

Files
<code>scripts/verify-phase8.ts</code>
<code>scripts/verify-phase9.ts</code>
<code>tests/auth.test.ts</code> , <code>tests/state-machine.test.ts</code>
<code>vitest.config.ts</code> , <code>tsconfig.check.json</code> , <code>.eslintrc.json</code>

4. Phase 6 and 7 Sync Logic

1. A job insert/status update or an evidence transition to `UPLOADED` invokes its database trigger before the outer transaction commits.
2. The trigger takes a per-recipient advisory transaction lock, creates recipient `sync_events` and `notifications` rows, and queues `pg_notify('networkpeer_sync', cursor)` in that transaction.
3. Commit makes the rows durable and delivers the PostgreSQL notification.
4. API instances with `REALTIME_ENABLED=true` receive it through `LISTEN networkpeer_sync` .
5. Each instance emits `sync:event` only to its local Socket.IO room, `user:<userId>` .
6. The browser invalidates cached jobs/notifications for a live event, then runs durable cursor recovery.
7. A tab stores its cursor in `sessionStorage` only after ordered REST pages have been applied; this prevents one tab from advancing another tab's checkpoint.
8. Recovery calls `GET /sync?cursor=<lastCursor>` repeatedly until `has_more` is `false` ; the advisory lock makes that per-recipient cursor

sequence commit-ordered and page-bounded.

9. If `PUSH_NOTIFICATIONS_ENABLED=true`, the Phase 9 relay enqueues durable sync cursors into BullMQ. The worker claims the row only when executing, sends FCM HTTP v1 notifications to registered device tokens, and records sent/skipped/retry state. PostgreSQL remains recoverable if Redis loses a job; retries stop after five attempts.
10. Wallet entries and notification read-state changes produce durable deltas without creating a duplicate inbox notification.
11. `GET /worker/sync` adds assigned-job snapshots/deltas and assignment removals to the same cursor protocol so a worker-local SQLite store can reconcile offline state.
12. Admin job overrides and worker suspension use the same job trigger path, so affected recipients receive durable events in addition to immutable audit rows; worker verification changes are audit-only until a profile-sync consumer is added.

The cursor is a PostgreSQL `BIGINT` serialized as a decimal string. Do not parse it with JavaScript `Number`; this avoids loss of precision over long-lived deployments.

5. Web Platform Code Map

5.1 Runtime and Shared Files

File	What it does in the website
<code>src/start.ts</code>	TanStack Start bootstrap and CSRF middleware for server functions.
<code>src/server.ts</code>	SSR error normalization for the Cloudflare/Nitro server entry.
<code>src/router.tsx</code>	Creates the TanStack Router and React Query client.
<code>src/routeTree.gen.ts</code>	Generated route manifest; do not edit manually.
<code>src/routes/__root.tsx</code>	Global QueryClient provider, toast host, and <code>RealtimeSyncBridge</code> .
<code>src/styles.css</code>	Global Tailwind/theme/layout styling.
<code>src/lib/api.ts</code>	Browser REST client, envelope parsing, bearer token refresh, paged

File	What it does in the website
	sync/notification methods.
<code>src/lib/auth-session.ts</code>	Browser-tab session store and React subscription hook.
<code>src/lib/auth-flow.js</code>	Phone formatting and basic OTP input helpers.
<code>src/lib/demo-jobs.ts</code>	Legacy local demo job persistence. Replace as client job APIs are wired.
<code>src/lib/mock-data.ts</code>	Legacy fixture data for unintegrated pages.
<code>src/lib/utils.ts</code>	CSS class merger and INR currency formatter.
<code>src/lib/error-capture.ts</code> , <code>src/lib/error-page.ts</code> , <code>src/lib/lovable-error-reporting.ts</code>	SSR/client error capture and fallback reporting.
<code>src/hooks/use-mobile.tsx</code>	Responsive viewport hook.
<code>public/</code> , <code>vite.config.ts</code> , <code>eslint.config.js</code> , <code>tsconfig.json</code>	Static assets and platform build, lint, and TypeScript configuration. The platform currently has no <code>test</code> or standalone <code>typecheck</code> npm script.

5.2 Components

File or group	What it renders
<code>src/components/realtime-sync-bridge.tsx</code>	Socket.IO connection, durable cursor reconciliation, React Query invalidation, live toast updates.
<code>src/components/auth/auth-ui.tsx</code>	Shared sign-in layout, fields, and visual controls.

File or group	What it renders
<code>src/components/shell/portal-shell.tsx</code>	Desktop/client/admin shell, navigation and notification badges.
<code>src/components/marketplace/primitives.tsx</code>	Marketplace cards, chips, maps, timelines, empty and stat states.
<code>src/components/theme-toggle.tsx</code>	Theme control.
<code>src/components/ui/accordion.tsx</code>	Accordion primitive.
<code>src/components/ui/alert-dialog.tsx</code>	Confirmation dialog primitive.
<code>src/components/ui/alert.tsx</code>	Alert primitive.
<code>src/components/ui/aspect-ratio.tsx</code>	Aspect-ratio primitive.
<code>src/components/ui/avatar.tsx</code>	Avatar primitive.
<code>src/components/ui/badge.tsx</code>	Badge primitive.
<code>src/components/ui/breadcrumb.tsx</code>	Breadcrumb primitive.
<code>src/components/ui/button.tsx</code>	Button primitive.
<code>src/components/ui/calendar.tsx</code>	Calendar primitive.
<code>src/components/ui/card.tsx</code>	Card primitive.
<code>src/components/ui/carousel.tsx</code>	Carousel primitive.
<code>src/components/ui/chart.tsx</code>	Chart primitive.
<code>src/components/ui/checkbox.tsx</code>	Checkbox primitive.
<code>src/components/ui/collapsible.tsx</code>	Collapsible primitive.
<code>src/components/ui/command.tsx</code>	Command palette primitive.
<code>src/components/ui/context-menu.tsx</code>	Context-menu primitive.
<code>src/components/ui/dialog.tsx</code>	Dialog primitive.
<code>src/components/ui/drawer.tsx</code>	Mobile drawer primitive.

File or group	What it renders
<code>src/components/ui/dropdown-menu.tsx</code>	Dropdown primitive.
<code>src/components/ui/form.tsx</code>	React Hook Form helpers.
<code>src/components/ui/hover-card.tsx</code>	Hover-card primitive.
<code>src/components/ui/input-otp.tsx</code>	OTP input primitive.
<code>src/components/ui/input.tsx</code>	Text input primitive.
<code>src/components/ui/label.tsx</code>	Label primitive.
<code>src/components/ui/menubar.tsx</code>	Menubar primitive.
<code>src/components/ui/navigation-menu.tsx</code>	Navigation-menu primitive.
<code>src/components/ui/pagination.tsx</code>	Pagination primitive.
<code>src/components/ui/popover.tsx</code>	Popover primitive.
<code>src/components/ui/progress.tsx</code>	Progress primitive.
<code>src/components/ui/radio-group.tsx</code>	Radio-group primitive.
<code>src/components/ui/resizable.tsx</code>	Resizable-panel primitive.
<code>src/components/ui/scroll-area.tsx</code>	Scroll-area primitive.
<code>src/components/ui/select.tsx</code>	Select primitive.
<code>src/components/ui/separator.tsx</code>	Separator primitive.
<code>src/components/ui/sheet.tsx</code>	Sheet primitive.
<code>src/components/ui/sidebar.tsx</code>	Sidebar primitive.
<code>src/components/ui/skeleton.tsx</code>	Loading skeleton primitive.
<code>src/components/ui/slider.tsx</code>	Slider primitive.
<code>src/components/ui/sonner.tsx</code>	Toast host wrapper.
<code>src/components/ui/switch.tsx</code>	Switch primitive.

File or group	What it renders
<code>src/components/ui/table.tsx</code>	Table primitive.
<code>src/components/ui/tabs.tsx</code>	Tabs primitive.
<code>src/components/ui/textarea.tsx</code>	Textarea primitive.
<code>src/components/ui/toggle-group.tsx</code>	Toggle-group primitive.
<code>src/components/ui/toggle.tsx</code>	Toggle primitive.
<code>src/components/ui/tooltip.tsx</code>	Tooltip primitive.

5.3 Route Files

Route file	Screen and current data source
<code>routes/index.tsx</code>	Public marketing/landing page.
<code>routes/auth.index.tsx</code>	Integrated phone OTP request screen.
<code>routes/auth.verify.tsx</code>	Integrated OTP verification and tab-session creation.
<code>routes/auth.admin.tsx</code>	Admin UI prototype. A migration-owner must first create an <code>ADMIN</code> account with <code>scripts/provision-admin.sql</code> ; the normal OTP flow then preserves that provisioned role.
<code>routes/client.tsx</code>	Client shell; live unread notification badge.
<code>routes/client.index.tsx</code>	Client dashboard; still demo job data.
<code>routes/client.jobs.index.tsx</code>	Client jobs list; still demo/local data.
<code>routes/client.jobs.new.tsx</code>	Client job creation form prototype.
<code>routes/client.jobs.\$jobId.tsx</code>	Client job detail prototype.
<code>routes/client.notifications.tsx</code>	Integrated Phase 6 notification inbox and read mutations.

Route file	Screen and current data source
<code>routes/client.review.\$jobId.tsx</code>	Evidence review prototype.
<code>routes/client.wallet.tsx</code>	Wallet prototype; financial phase is not implemented.
<code>routes/worker.tsx</code>	Worker mobile shell.
<code>routes/worker.index.tsx</code>	Nearby job discovery prototype using mock data.
<code>routes/worker.job.\$jobId.tsx</code>	Job detail/acceptance prototype using mock data.
<code>routes/worker.task.\$jobId.tsx</code>	Evidence capture prototype; does not yet access browser camera/S3.
<code>routes/worker.profile.tsx</code>	Worker profile prototype.
<code>routes/worker.wallet.tsx</code>	Worker wallet prototype.
<code>routes/admin.tsx</code>	Admin shell.
<code>routes/admin.index.tsx</code>	Admin dashboard prototype.
<code>routes/admin.analytics.tsx</code>	Analytics prototype.
<code>routes/admin.clients.tsx</code>	Client management prototype.
<code>routes/admin.jobs.tsx</code>	Job management prototype.
<code>routes/admin.payments.tsx</code>	Payments prototype.
<code>routes/admin.reviews.tsx</code>	Review prototype.
<code>routes/admin.settings.tsx</code>	Settings prototype.
<code>routes/admin.workers.tsx</code>	Worker verification UI prototype.

6. Frontend and Backend Integration Plan

Step 1: Configure local development

1. Start Postgres and Redis with `docker compose up -d` from the backend.
2. Run backend migrations with `npm run migrate`.
3. Copy backend `.env.example` to `.env` ; use `CORS_ORIGINS=http://localhost:5173` if Vite uses its standard port.

4. Copy frontend `.env.example` to `.env` ; set `VITE_API_BASE_URL=http://localhost:3000/api/v1` .
5. Start backend `npm run dev` and web platform `npm run dev` .

Step 2: Authentication and session

1. `/auth` calls `POST /auth/otp/request` .
2. `/auth/verify` calls `POST /auth/otp/verify` and stores the returned token pair in browser `sessionStorage` .
3. `src/lib/api.ts` attaches the access token, refreshes once on a 401, and clears the session on failed refresh.
4. Add a router guard before integrating private pages broadly so `/client` , `/worker` , and `/admin` redirect unauthenticated users. No private route is guarded today.
5. Provision initial administrators only with `psql "$DATABASE_URL" -f scripts/provision-admin.sql` as the migration owner. The OTP endpoint does not accept a role escalation request for an existing user.
6. Migrate refresh tokens to secure HttpOnly cookies when the backend session transport is redesigned; current bearer refresh tokens are intentionally tab-scoped but remain exposed to browser JavaScript.

Step 3: Realtime and notification inbox

1. `RealtimeSyncBridge` opens Socket.IO at `/api/v1/realtime` with `auth.token` .
2. It stores a per-tab durable sync checkpoint in `sessionStorage` only after ordered REST recovery; a live Socket.IO event never advances that checkpoint by itself.
3. Every `sync:event` invalidates the notifications, client jobs, and worker jobs query keys.
4. On `sync:ready` or reconnect it calls `/sync` until `has_more` is false.
5. `/client/notifications` displays the newest 100 rows from `GET /notifications` , marks individual rows read, and marks all read. It does not yet paginate older rows; read-state changes now generate durable cross-tab/device sync deltas.

Step 4: Replace client job mocks

1. Add `api.createJob` , `api.listClientJobs` , `api.getClientJob` , and `api.cancelClientJob` methods to `src/lib/api.ts` .
2. Map the form's whole-rupee amount to integer cents (`₹780` becomes `budget_cents: 78000`) and send `currency: "INR"` ; geocode its address to the API's required coordinates.

3. Replace `client.jobs.index.tsx` local state with `useQuery(["client", "jobs"])`.
4. Replace client detail and cancellation UI with the scoped backend routes.
5. After creation, present the `FUNDING` job and call `POST /client/jobs/:jobId/fund` with a stable idempotency key. Do not expose a job to worker discovery until its provider webhook succeeds.
6. Defer current form-only fields that the API does not accept: attachments, priority, estimated duration, and task-specific media requirements.
7. Remove demo persistence only after all job views use real data.

Step 5: Replace worker discovery and work mocks

1. Acquire browser geolocation only after explicit consent and call `POST /worker/location`; nearby discovery and job acceptance use this recent server-held location rather than arbitrary coordinates on each search.
2. Map public nearby result fields to `worker.index.tsx`; do not show exact location before assignment.
3. Replace mock acceptance with `POST /worker/jobs/:jobId/accept` and invalidate worker/client job queries.
4. Use `POST /work/status` to reach `IN_PROGRESS`; evidence endpoints require a verified, assigned worker and this job state.
5. In `worker.task.$jobId.tsx`, use `MediaRecorder` /camera capture, calculate SHA-256, request `/work/upload-url`, retain the returned `media_id`, submit the multipart POST fields to S3, call `/work/evidence`, then call `/work/submit`.
6. Submission requires confirmed `UPLOADED / VERIFIED` evidence for every required subtask, including checksum, ETag, and non- `null` S3 version ID. Location is optional and only bounds-validated; there is no on-site attestation or per-subtask media-type policy yet.
7. Treat all S3 keys as opaque. Never persist or derive them in browser code beyond the one signed form submission.
8. Do not wire `client.review.$jobId.tsx` to real data until client-scoped evidence list/download and approve/reject/completion APIs exist.

Step 6: Add worker local sync

1. Keep a local SQLite store keyed by job UUID and the opaque decimal-string sync cursor.
2. On app start, reconnect, or a `sync:event`, call `GET /worker/sync?cursor=<cursor>` until `has_more` is false.
3. Apply `jobs` snapshots/deltas and `removed_job_ids` atomically with the returned `events`, then persist the returned cursor. Never convert the cursor to a JavaScript number.

4. Treat REST job details as authoritative when a worker opens a job; do not persist client identity or exact location for unassigned jobs.

Step 7: Browser push notifications

1. Create a Firebase web app and service worker in the frontend.
2. Ask for browser notification permission only from a user action.
3. Obtain an FCM token and call `POST /notifications/devices` with platform `WEB`.
4. Keep in-app sync enabled even with browser notifications disabled; sync is the durable source of truth.
5. On token refresh/logout, register the new token or add a token removal endpoint before production rollout.

Step 8: Replace admin prototypes

1. Use only an authenticated `ADMIN` session to call the backoffice endpoints. The backend independently reloads the active, verified role before every request.
2. Populate job recovery controls with `POST /admin/jobs/:jobId/override`. `STATUS` may only follow the canonical lifecycle; `REASSIGN` requires an eligible nearby verified worker. Funded completion and cancellation cannot bypass escrow settlement or refund requirements. All actions require a 3-2,000-character reason and return an `audit_id`.
3. Populate client/worker management from `GET /admin/users`, use `POST /admin/users/:userId/suspend` for non-admin accounts, and retain the returned audit identifier in the UI confirmation.
4. Use `PATCH /admin/workers/:workerId/verification` for verification changes, `GET /admin/audit-log` for keyset-paginated audit history, and `GET /admin/analytics` for current operational/ledger aggregates.
5. Analytics are grouped by currency and use the authoritative completed ledger-account side of each journal. Do not present frozen escrow as settled revenue.

Step 9: Integrate funding, approval, and wallets

1. Show newly created jobs as payment-pending. Call `POST /client/jobs/:jobId/fund` once per stable client idempotency key and use the returned provider client secret only with the provider SDK.
2. Treat `POST /webhooks/payments` as the only source that moves a funding operation to `SUCCEEDED`; the browser must refresh its client job list rather than infer payment success locally.
3. After submitted work review, call `POST /client/jobs/:jobId/approve` with a stable idempotency key. Retrying the same request returns the

original settlement and current payout operation state.

4. Read `GET /client/wallet` and `GET /worker/wallet` as account projections. Never calculate wallet balances, platform fees, or payouts in browser code.
5. Surface `FROZEN` escrow as an operational-support state. Do not offer cancellation or completion controls for frozen work until an audited refund/dispute resolution API exists.

Step 10: Operate background workers

1. Keep `BACKGROUND_QUEUES_ENABLED=true` to run BullMQ workers inside the API process, or set it to `false` on API replicas and run `npm run worker` on dedicated worker replicas.
2. Redis is queue transport, not the durable source of truth. The worker sweeps PostgreSQL `media_processing_outbox` and pending `sync_events`, so an enqueue failure or Redis restart only delays delivery.
3. Media processing re-headers the exact accepted S3 `VersionId` and verifies size, MIME type, SHA-256 checksum, and ETag. It records processing completion without changing evidence acceptance status.
4. Push processing claims one sync cursor immediately before delivery. A transient gateway failure releases the durable claim with a PostgreSQL `push_not_before_at` due time set from FCM `Retry-After` (never less than one minute). BullMQ may retry its transport job early, but the durable claim query rejects that cursor until due and the database sweep re-enqueues it afterward.

7. Deployment Plan

7.1 Provision external services

1. PostgreSQL 16 with PostGIS, automated backups, TLS, and a migration-owner role.
2. Redis with TLS and persistence appropriate for refresh sessions/rate limits.
3. Private versioned S3 evidence bucket with Block Public Access, default encryption, `PutObjectTagging`, browser-origin CORS for the presigned POST flow, and a lifecycle rule that expires `networkpeer-evidence-state=pending` objects. The API signing/verification role needs the configured `PutObject`, object metadata read/ `HeadObject`, and `PutObjectTagging` permissions; browsers use the returned presigned form rather than IAM credentials.
4. Firebase service account with FCM HTTP v1 permission if push is enabled.

5. A WebSocket-capable API load balancer/proxy. Preserve Upgrade headers. `src/lib/api.ts` connects Socket.IO directly to the API origin; the backend enforces WebSocket-only transport. Set `TRUST_PROXY_CIDRS` only to the proxy's actual IP/CIDR ranges, never to a broad internet range.
6. Stripe account, restricted server secret, webhook endpoint secret, and connected worker recipient accounts if production payouts are enabled. Configure Stripe to deliver payment-intent and Connect transfer events to the API webhook endpoint.
7. Redis persistence and memory capacity for BullMQ. Queue delivery is recoverable from PostgreSQL outboxes, but Redis availability still determines processing latency.

7.2 Deploy the backend

1. Build and test: `npm ci`, `npm run typecheck`, `npm run lint`, `npm test`, `npm run build`, `npm audit`.
2. Run `npm run migrate` once with the migration owner, never the runtime application role.
3. Run `psql "$DATABASE_URL" -f scripts/provision-app-role.sql` as the migration owner. It provisions `networkpeer_app`, `networkpeer_admin_api`, `networkpeer_media_verifier`, and `networkpeer_financial_api`; the script prompts interactively for their passwords. Do not put credentials in source or deployment logs.
4. Provision the initial backoffice account once with `psql "$DATABASE_URL" -f scripts/provision-admin.sql` as the migration owner. Record the approved phone number through the normal operational secret process, not source control.
5. Configure production values: `NODE_ENV=production`, `DATABASE_URL` with `sslmode=require`, dedicated `DATABASE_ADMIN_URL`, `DATABASE_MEDIA_VERIFIER_URL`, and `DATABASE_FINANCIAL_URL`, `REDIS_URL` with `rediss://`, distinct non-placeholder JWT secrets, `SMS_PROVIDER=twilio`, `OTP_ECHO_IN_RESPONSE=false`, `LOG_PRETTY=false`, non-placeholder S3 bucket/credentials, explicit `HTTPS_CORS_ORIGINS`, explicit `TRUST_PROXY_CIDRS`, `PAYMENT_GATEWAY=stripe`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, and optional Firebase values.
6. Set `PUSH_NOTIFICATIONS_ENABLED=true` only when all Firebase values and device registration are ready. Set `BACKGROUND_QUEUES_ENABLED=false` on API-only replicas when using dedicated worker replicas.
7. Start the API and, when separated, `npm run worker`. Production startup verifies S3 public-access blocks, versioning, and default encryption before accepting traffic.
8. Verify `/api/v1/live`, `/api/v1/health`, one authenticated Socket.IO connection, and `npm run verify:phase6`, `npm run verify:phase7`, `npm`

run `verify:phase8`, and `npm run verify:phase9` against a non-production environment.

7.3 Deploy the frontend

1. Run `npm ci`, `npm run build`, `npm run lint`, and `npm audit` in `NetworkPeer-platform-main`.
2. Set `VITE_API_BASE_URL=https://api.example.com/api/v1` and `VITE_API_PREFIX=/api/v1` at build time.
3. Deploy the generated TanStack/Nitro output to a WebSocket-compatible platform. The current Nitro preset is Cloudflare module; validate WebSocket proxy behavior in the selected platform before production.
4. Add the exact deployed web origin to backend `CORS_ORIGINS` and redeploy/restart backend config.
5. Browser smoke test: request OTP, verify OTP, see the client notification badge, create or transition test data through REST with an administratively pre-verified available worker, observe the client realtime notification, refresh the tab, and confirm cursor recovery. The current worker/client UI cannot yet perform a real acceptance flow.

7.4 Rollback and Operations

1. Database migrations are forward-only. Do not delete `schema_migrations` records to roll back.
2. Roll back application artifacts only when the schema remains compatible; otherwise ship a forward repair migration.
3. Monitor API 5xx/429 rates, Redis availability, PostgreSQL listener connection health, Socket.IO connection count, sync cursor lag, notification unread growth, and FCM push failures.
4. Keep S3 lifecycle rules and versioning enabled during rollback; accepted evidence depends on stored version IDs.
5. Rotate JWT, Twilio, S3, Firebase, and database credentials through the platform secret manager, not `.env` files committed to source control.

8. Current Integration Status and Next Work

- Complete now: backend Phase 9, browser OTP session, API client, Socket.IO bridge, ordered per-tab cursor recovery, unread badge, client notification inbox, device token API, optional FCM sender, worker local-sync API, audited admin APIs, webhook-settled escrow/payout/wallet APIs, and durable media/push queue workers.
- Intentionally pending: private-route authorization, real client job screens, real worker discovery/acceptance screens, browser media capture/S3 upload UI, client evidence review UI, Firebase web service worker, admin

UI integration, Stripe recipient onboarding, automatic refund/dispute resolution for frozen funds, and richer media inspection/transcoding policy.

- The next safe implementation order is private-route guards, client jobs plus funding, worker discovery/acceptance, worker evidence capture/local SQLite sync, client review/approval/wallet screens, Firebase browser registration, admin UI integration, Stripe recipient onboarding/refund operations, media policy workers, and observability.